Maren

CONFIDENTIAL

Maren Finance
Progress & Evidence

What has been built so far, deployed where, proven how, and the exact commands and transaction hashes that let you check every claim without trusting us.

DOCUMENT	DATE	VERSION	PREPARED FOR
MRN-100	17 August 2026	1.1	Mezo Ecosystem

DOCUMENT	DATE	CLASSIFICATION	VERSION
MRN-100	17 August 2026	Confidential	1.1

Maren Finance — Progress & Evidence

What has been built so far, deployed where, proven how, and the exact commands and transaction hashes that let you check every claim without trusting us.

CONTENTS

1 What Maren is, in one paragraph 2
2 What was built 2
3 Deployed contracts, Mezo matsnet (31611) 2
4 Exercised on-chain, not just deployed 3
5 Test evidence 3
6 The product, as it runs today 4
7 Findings from reading the chain 6
8 Where the numbers a user sees come from 6
9 Not done, stated plainly 6
10 Reproduce every number 7

1

What Maren is, in one paragraph

Maren is a consumer borrowing and payments app for Mezo, on web and iOS. Users deposit Bitcoin, borrow MUSD against it from as little as 0.001 BTC, and spend it: pay a `@name`, share a payment link, request money. It is built for Mezo and deployed nowhere else. Everything below is what exists today, with the evidence.

2

What was built

The codebase is a single monorepo with five parts, sharing one SDK so that every number a user sees is computed in exactly one place.

COMPONENT	WHAT IT IS
Contracts	Five Solidity contracts on Mezo matsnet: a pooled collateral vault, a payments contract, an on-chain name registry, an ERC-4626 yield vault, and an ERC-2771 relay hub. Foundry, Solidity 0.8.28, OpenZeppelin v5
SDK	One TypeScript package holding every protocol constant, address, and piece of position math, verified against the chain and consumed by both apps. A borrow limit shown on iOS is the same function call as the one on web
Web app	Next.js. Portfolio, Vault, Pay, Earn, Activity. Reads live contract state; the write path is wired and exercised on testnet
iOS app	Expo, 18 screens, verified in the simulator including the degraded states (stale oracle, RPC failure)
Relayer	A gasless transaction service: verifies a user's signed request, simulates it, submits it, and settles gas from its own key. Built and fully tested, not yet hosted

Two design decisions are worth naming because they shape everything else:

1. **Preflight before signature.** The interface evaluates every rule the deployed contract enforces, in the contract's own order, before a control is enabled. An action Maren offers is an action that clears; a refusal states the same reason the revert would have given, with live numbers. Nobody pays gas to discover a "no".
2. **Nothing rendered that cannot be proven.** Activity is read from contract logs; every row carries a real transaction hash linked to the explorer. There is no sample data anywhere in the product, and the empty states say precisely what was scanned before concluding there was nothing.

3

Deployed contracts, Mezo matsnet (31611)

Five contracts, all source-verified on Blockscout, deployed from `0x54196397D6f01b48739dB3957257cB33cD341885` in 11 transactions costing 11,688,063 gas in total (about 4 nano-BTC).

CONTRACT	ADDRESS	DEPLOY TRANSACTION
MarenVault	`0x46252B00AB59384518d9dB6D1f210ffF1221645b`	`0x2525...9622`
MarenPay	`0xe475Ff919D54E055809f516E3bed8a5c7a101836`	`0xa91c...c807`
MarenRelayHub	`0x22cC1Adfb0C91D9359746dE3F7E8cB7F2B8eA98C`	`0xc1fd...db74`
MarenRegistry	`0x9e123Cdd0CD524812CF9F552d3FAB13d8143b77E`	`0x1a48...eae4`
MarenEarn	`0xE3B02b215085B99Ce61E0F5ea9763Ac637aa7826`	`0x47d7...46c3`

Full 64-character hashes for every transaction in this document are recorded in the repository's deployment log and resolve on `explorer.test.mezo.org`. MarenRelayHub extends the ERC-2771

forwarder and is the single trusted forwarder; `MarenPay.trustedForwarder()` returns its address on-chain.

4

Exercised on-chain, not just deployed

Deployment proves compilation. The claims below prove operation, each with its transaction.

- **A pooled MUSD trove is open and active inside Mezo's own TroveManager** (`openPooledTrove` , 762,931 gas, tx `0x13ea...93f8`), currently at 0.069 BTC of collateral against 2,001.9 MUSD of debt: an ICR of **220%**, with borrowing enabled. Verify with one call: `cast call 0xE47c80e8c23f6B4A1aE41c34837a0599D5D16bb0 "getCurrentICR(address,uint256)(uint256)" 0x46252B00AB59384518d9dB6D1f21`
- **A live borrow is offered and correctly bounded.** From the funded test account, a 30 MUSD borrow simulates clean and a 40 MUSD borrow reverts with `InsufficientCollateralRatio` at the account's own 180% floor. The interface offers exactly the range the contract accepts, computed before any signature is requested.
- **The `@sele` handle is registered through commit-reveal** (commit `0x91b7...ad04` , reveal `0x0484...6581` , 119,726 gas). `resolve("sele")` returns the deployer address on-chain. Commit-reveal exists so a name cannot be front-run in the mempool.
- **User flows executed on the live chain:** Bitcoin deposits (`0xda9a...602b` among others), a withdrawal, keeper collateral top-ups (`0xb3b9...c7f2` , `0xdc00...3cb8`), and an on-chain payment request of 0.50 MUSD. Each is a row in the app's Activity screen with its hash linked to the explorer.
- **One honest failure, kept on the record.** An early `deleverage` call (`0x71ad...c54d`) reported success while doing nothing, because at the protocol's minimum debt there was no repayment room and the no-op path returned silently. It was found by reading state after the call rather than trusting the receipt, and it is being fixed by making the silent path loud. It is included here because the process that caught it is the same one behind every other number on this page.

5

Test evidence

SUITE	COUNT	RUNS AGAINST
Contract unit and property tests	236	Foundry
Fork tests	8	The real MUSD contracts on matsnet
Relayer	214	Four adversarial review rounds, each of which found and fixed a real defect
Web behaviour suite	87	The built app against live matsnet, through an injected wallet that structurally cannot sign

Three of these deserve a sentence each.

The fork tests run Maren's contracts against the actual deployed MUSD system on matsnet rather than a mock, so a divergence between MUSD's documentation and its bytecode (there are several, listed below) breaks a test instead of a user.

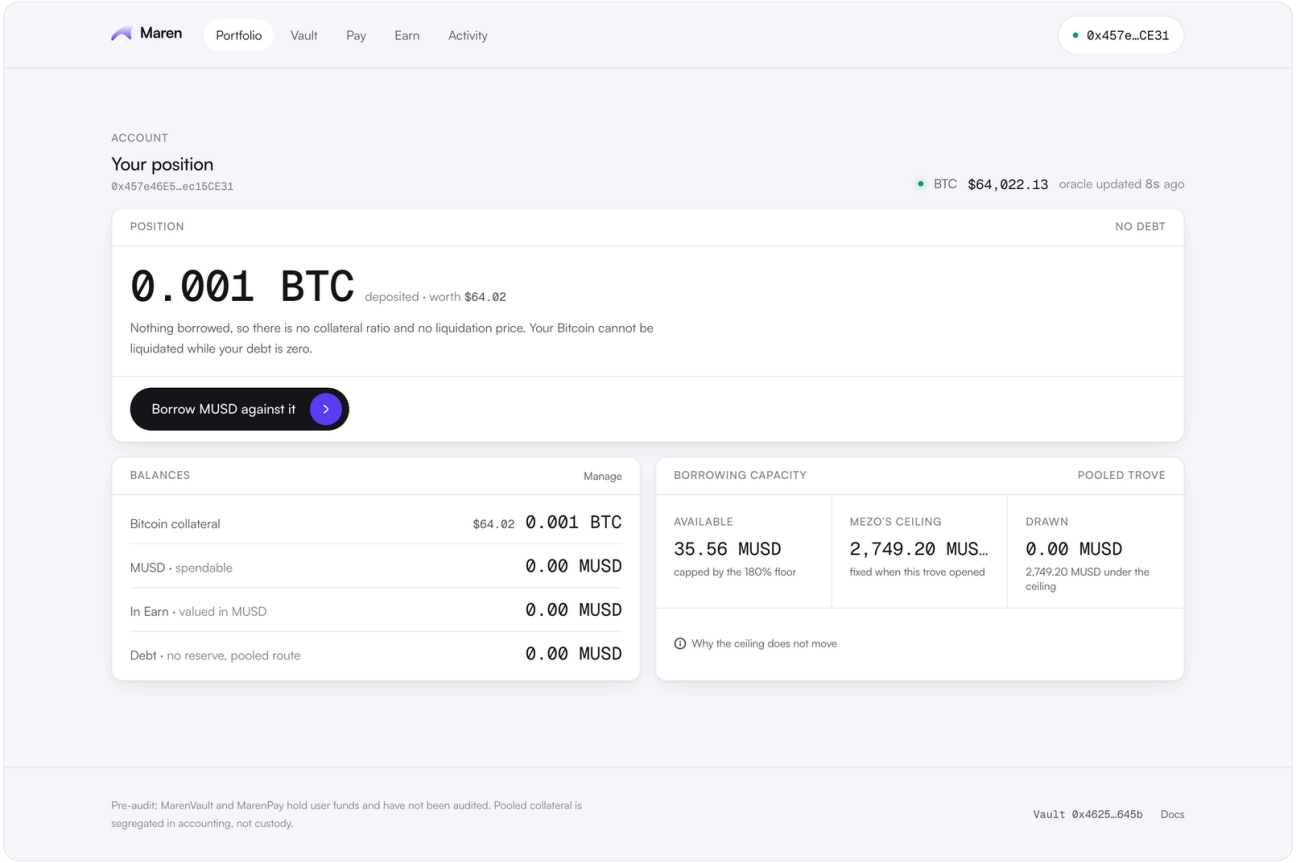
The relayer's four adversarial rounds were structured attacks on our own design. Round two found a budget race that allowed 52x overspend; round three found that the fix created a denial-of-service path; round four found the CI had never actually exercised the relayer's code. Each round's finding is fixed and regression-tested.

The behaviour suite drives the real interface against the real chain with a read-only signer, asserts that every offered action matches what the deployed contract would accept, and proves no signature is ever requested: the account nonce is read before and after as the final check.

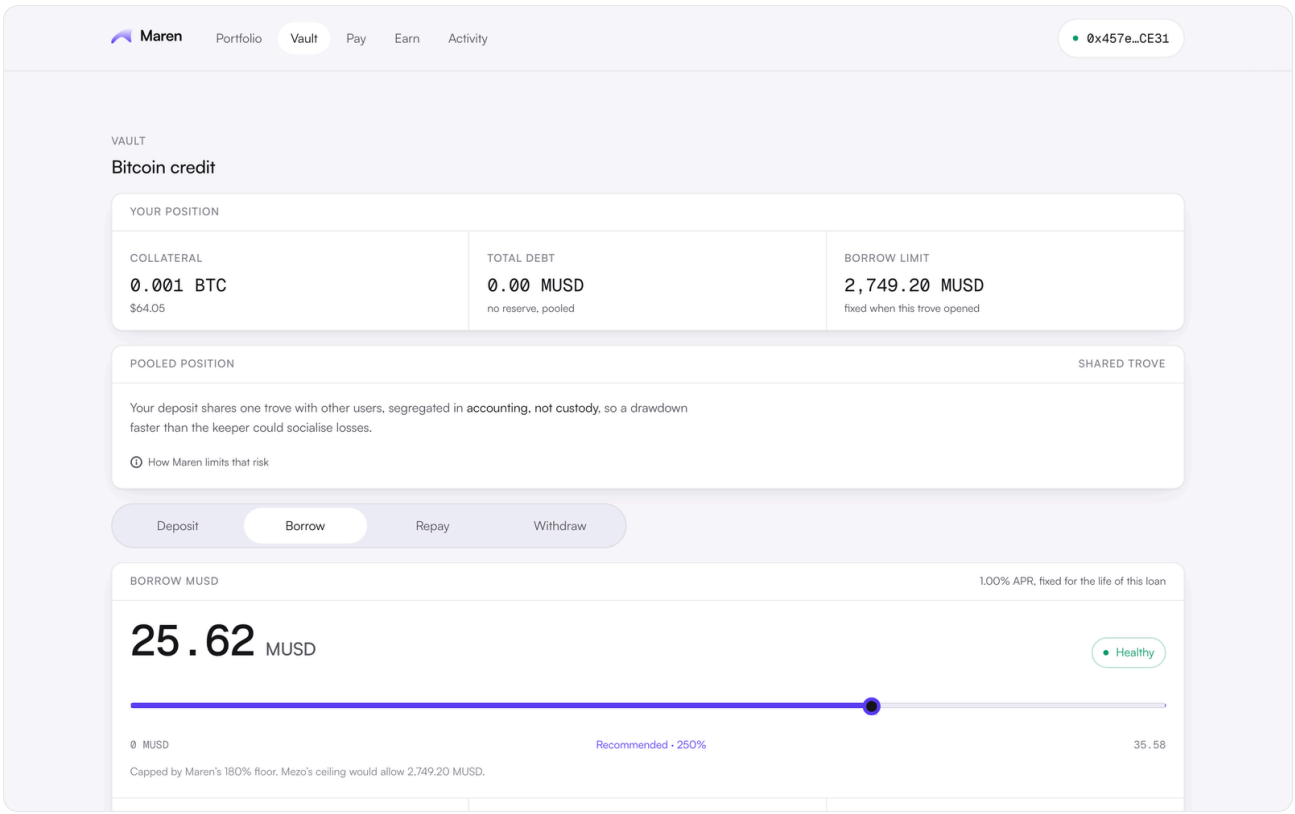
6

The product, as it runs today

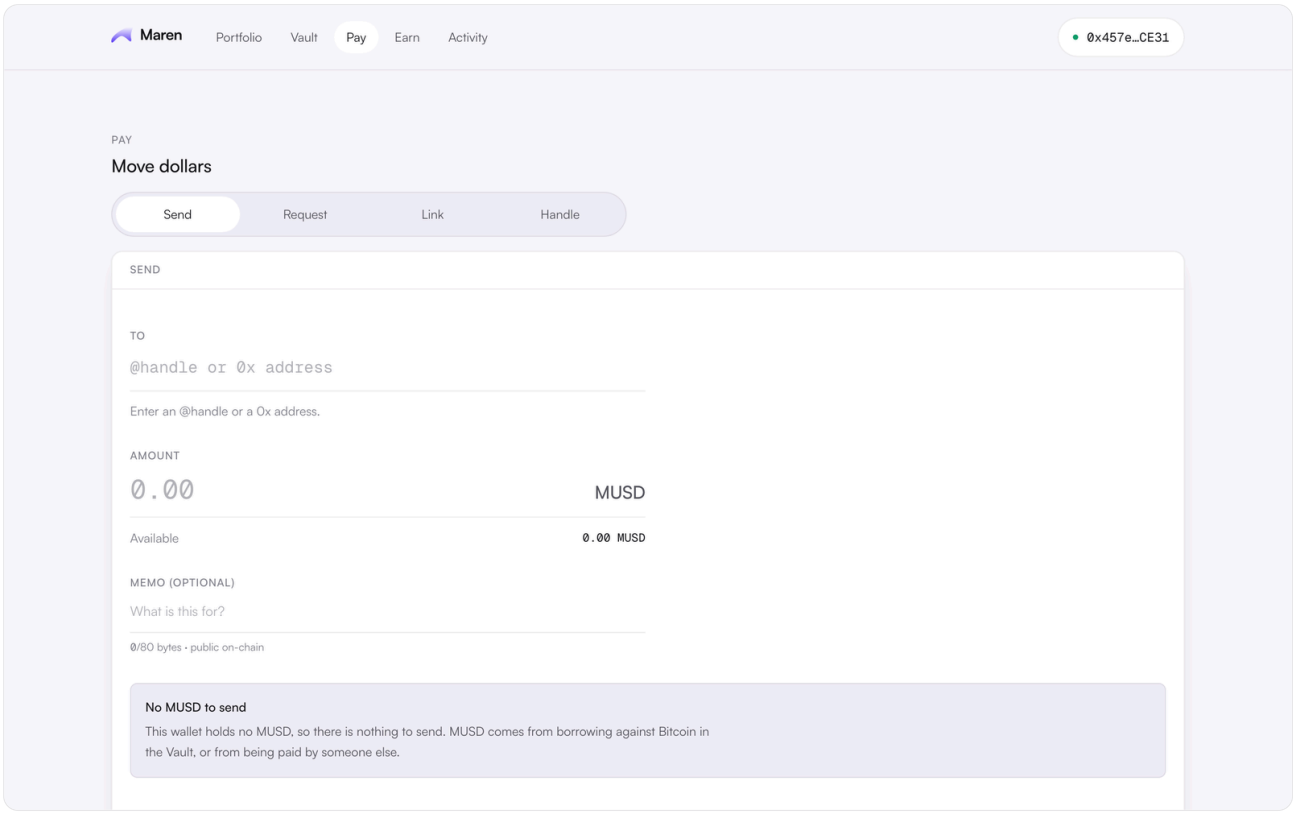
Unretouched captures of the web app in a read-only session against live matsnet, 17 August 2026. Every figure shown is a contract read.



Portfolio. A funded account with no debt: 0.001 BTC deposited, and the interface states plainly that nothing can be liquidated while debt is zero.



Vault. Borrowing live against the pooled trove: the slider is bounded by the same computation the contract enforces, with the recommended 250% marked.



Pay. MUSD moves against @handles or raw addresses, with an on-chain memo. An empty wallet is told where MUSD comes from rather than just refused.

7

Findings from reading the chain

The build began with several days of reading Mezo's deployed bytecode rather than its documentation. That produced findings useful to Mezo regardless of anything else:

1. **`viem`'s Mezo chain definition** ships a default RPC that does not resolve and is missing `multicall3` on mainnet, which silently degrades batched reads for every wagmi user on the chain. A fix is drafted.
2. **The subgraph documentation** says `network: mezo-mainnet`; Goldsky's actual slug is `mezo`, so a manifest copied from the docs is rejected at deploy. Fix drafted.
3. **Three RPC hosts in the developer docs do not resolve.** Fix drafted.
4. **Two places the MUSD docs and deployed bytecode disagree:** the borrower redemption-fee waiver does not appear in `TroveManager`, and the refinance fee computes to roughly 0.02% rather than the documented 0.1%.
5. **ERC-4337 is not viable on Mezo today.** EntryPoint v0.7 is deployed byte-identical to Ethereum L1 but has processed zero UserOperations per the block explorer index, and the RPCs do not implement `debug_traceCall`, so no compliant bundler can run. Maren uses EIP-2612 permits with ERC-2771 relaying instead, and the chain accepts EIP-7702 type-4 transactions as the upgrade path.
6. **The BTC/USD oracle is Mezo's own consensus precompile**, not Chainlink or Pyth, and it reverts above 60 seconds of staleness. Pyth is deployed but weeks stale. Both apps therefore carry an explicit "price unavailable" state and never render a stale price as live.
7. **A sizing result worth passing on:** a pooled vault seeded at the protocol minimum can never serve a borrow, because MUSD's 1,800 minimum net debt pins the seed below its own borrowing floor. The minimum viable pooled vault is larger than the minimum trove. We hit this in practice and resolved it by sizing the seed to policy; the pooled trove's move from 153% to 220% ICR is on-chain in two transactions.

8

Where the numbers a user sees come from

Every protocol parameter in the product (the 110% liquidation threshold, the 1,800 MUSD minimum, the 1.00% fixed simple interest, the 0.1% origination fee, the per-trove borrowing ceiling that adding collateral does not raise) was read from the deployed contracts and is recorded in one SDK module with its verification date. Seven documented ways MUSD diverges from the Liquity v1 codebase it descends from are recorded in the same place, because each one breaks a naively ported frontend.

9

Not done, stated plainly

No mainnet deployment yet: it is gated on an independent audit, deliberately. No audit yet. The gasless relayer is built and fully tested but not yet hosted, so today users submit their own transactions and the app says so on every screen. Bank cashout and merchant payments are design work, not code. Nothing in this document overstates that.

10

Reproduce every number

All chain figures above re-read in one pass with `cast` and `curl` against `https://rpc.test.mezo.org` (matsnet, 31611) and `https://jsonrpc-mezo.boar.network` (mainnet, 31612). The addresses and hashes in this document are sufficient to verify independently; the repository additionally carries a verification script that prints each figure beside the command that produced it.

Every chain figure in this document was read live from Mezo RPCs on 17 August 2026, and each one is paired with the command, address, or transaction hash that reproduces it. Counts that drift daily are quoted at the precision at which they hold. Where Mezo's published documentation and the deployed bytecode disagree, this document follows the bytecode and says so.